THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 3

Databricks Workspace, Notebooks & Clusters — The Complete Guide

SECTION 01

The Workspace — Your Home Base

The Workspace is the top-level environment in Databricks where all your assets live: notebooks, folders, libraries, dashboards, and experiments. Think of it as the organizational container for everything your team builds.

What Lives Inside a Workspace

- Folders and notebooks, organized per user, per team, or per project.
- Repos, which connect the workspace to Git providers like GitHub or Azure DevOps for version control.
- Workspace-level permissions, controlling who can view, edit, or run specific assets.
- Shared resources such as clusters, jobs, and dashboards that a whole team can access.

CORE CONCEPT

A Databricks Workspace is not a compute resource itself. It is an organizational and access-control layer sitting in the Control Plane. The actual work happens once a notebook is attached to a cluster.

SECTION 02

Notebooks — Where You Write Code

A notebook is an interactive, cell-based coding environment. Each cell can be run independently, and results appear directly below the cell, which makes notebooks ideal for exploration, debugging, and iterative development.

Multi-Language Support

Databricks Notebooks support Python, SQL, Scala, and R within the same notebook. The default language is set when the notebook is created, but any individual cell can override it using a magic command at the top of the cell.

Common Magic Commands

Magic Command	What It Does
%python	Switches the current cell to Python, regardless of the notebook's default language.
%sql	Runs the cell contents as a SQL query.
%scala	Switches the current cell to Scala.
%r	Switches the current cell to R.
%md	Renders the cell as Markdown text, useful for documentation within the notebook.
%fs	Runs filesystem commands against the Databricks file system.
%sh	Executes shell commands directly on the cluster's driver node.
%pip	Installs Python packages scoped to the current notebook session.

```
# Example: mixing languages in one notebook

%python
df = spark.read.table("sales.transactions")
df.createOrReplaceTempView("transactions_view")

%sql
SELECT region, SUM(amount) AS total_sales
FROM transactions_view
GROUP BY region
```

GOOD PRACTICE

Since clusters are ephemeral, any packages installed with %pip disappear once the cluster restarts. Teams typically track dependencies in a requirements file or cluster-scoped library configuration instead of relying on ad-hoc installs.

SECTION 03

Clusters — Where Code Actually Runs

A cluster is a set of computation resources and configurations Databricks provisions to run your notebooks and jobs. Every cluster has one driver node, and optionally one or more worker nodes, depending on the workload.

Driver Node vs Worker Nodes

- Driver node: coordinates the work, maintains the SparkContext, and collects final results.
- Worker nodes: execute the actual distributed tasks assigned by the driver.

Two Types of Clusters

- All-Purpose Clusters: created via the UI, CLI, or REST API, used for collaborative interactive analysis. They can be manually started, stopped, and restarted, and shared across multiple users.
- Job Clusters: created automatically by the Databricks job scheduler when a job runs, and terminated automatically once the job completes. They cannot be manually restarted.

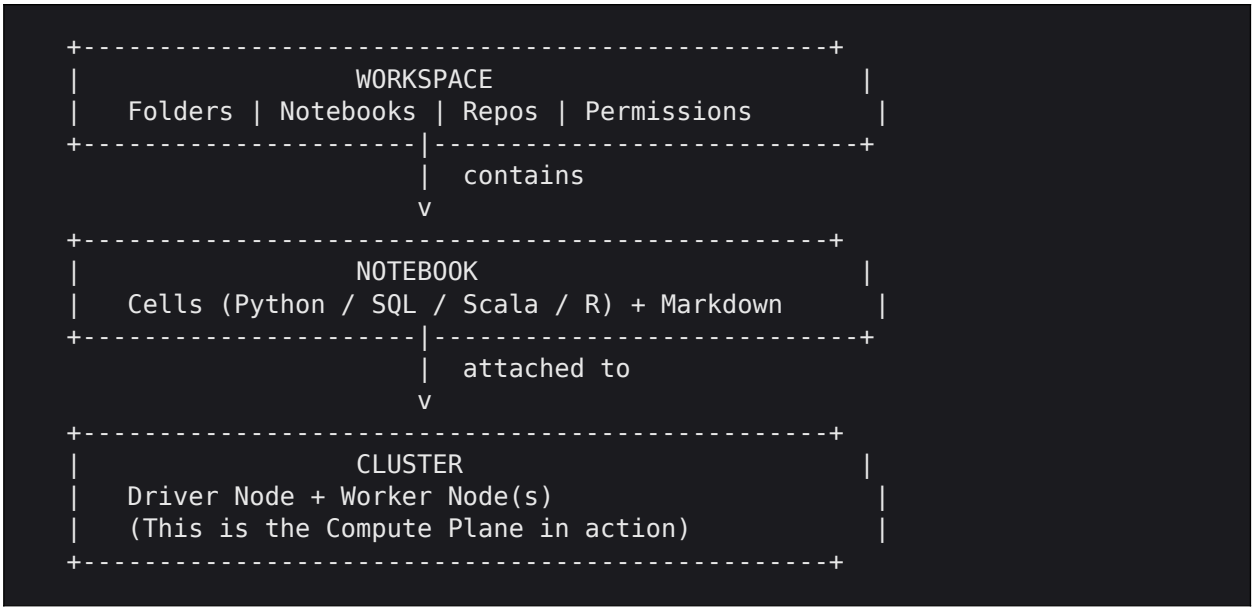
RULE OF THUMB

Use all-purpose clusters during development and exploration. Switch to job clusters for scheduled, automated production pipelines, since they are cheaper, isolated per run, and terminate automatically.

SECTION 04

How the Three Pieces Connect

A simplified view of how a Workspace, Notebook, and Cluster relate to one another:



REMEMBER THIS FOR INTERVIEWS

A Notebook defines what to run. A Cluster defines where and how it actually runs. Without an attached, running cluster, a notebook's code is inert text; nothing executes until compute is available.

SECTION 05

Key Cluster Configuration Settings

Setting	What It Controls
Databricks Runtime Version	The software stack: Spark version, Python/Scala versions, and pre-installed libraries. LTS versions are recommended for production.
Node Type	The VM instance type used for driver and worker nodes, affecting memory, CPU, and cost.
Autoscaling	Automatically adjusts the number of worker nodes based on workload demand, within a configured min and max range.
Auto-Termination	Automatically shuts down an idle all-purpose cluster after a specified period of inactivity, to control cost.
Access Mode	Controls whether a cluster is shared across users or dedicated to a single user, and affects governance features available.

SECTION 06

All-Purpose vs Job Clusters

Aspect	All-Purpose Cluster	Job Cluster
Created By	Manually, via UI, CLI, or REST API	Automatically, by the job scheduler
Lifecycle	Manually started, stopped, restarted	Auto-terminates when the job ends
Typical Use	Interactive development, collaboration	Scheduled, automated production jobs
Sharing	Can be shared across multiple users	Dedicated to a single job run
Cost Behavior	Runs until manually stopped or auto-terminated	Runs only for the job's duration

SECTION 07

Glossary — Day 3 Terms

Term	Definition
Workspace	The organizational environment holding notebooks, folders, repos, and permissions.
Notebook	A cell-based interactive coding environment supporting Python, SQL, Scala, and R.
Cluster	A set of provisioned compute resources, with a driver node and optional worker nodes, that runs notebook and job code.
Driver Node	The node that coordinates work, holds the SparkContext, and collects results.
Worker Node	A node that executes the distributed tasks assigned to it by the driver.
All-Purpose Cluster	A manually managed cluster used for interactive, collaborative development.
Job Cluster	An ephemeral cluster automatically created and destroyed for a single scheduled job run.
Magic Command	A notebook directive, such as %sql or %python, that changes how a cell is interpreted.

SECTION 08

Interview Questions & Answers

Q1. What is the difference between a Workspace and a Cluster?

A. A Workspace is an organizational environment holding notebooks, folders, and permissions; it does not run code. A Cluster is the actual compute resource, with a driver and worker nodes, that executes the code inside a notebook.

Q2. What is the difference between an all-purpose cluster and a job cluster?

A. An all-purpose cluster is manually created and managed, used for interactive and collaborative work, and can be shared across users. A job cluster is created automatically by the job scheduler for a single automated run and terminates once that job finishes.

Q3. Can a single notebook use multiple languages?

A. Yes. A notebook has a default language, but any cell can override it using a magic command such as %sql, %python, %scala, or %r.

Q4. What is the role of the driver node in a cluster?

A. The driver node coordinates the overall Spark job, maintains the SparkContext, schedules tasks across worker nodes, and collects the final results.

Q5. Why would a production pipeline use a job cluster instead of an all-purpose cluster?

A. Job clusters are cost-efficient since they exist only for the duration of the job, they are isolated per run reducing interference between jobs, and they terminate automatically, avoiding idle cluster costs.

Q6. What does auto-termination do, and why does it matter?

A. Auto-termination automatically shuts down an idle all-purpose cluster after a set period of inactivity, which helps control cloud compute costs, especially important for shared development clusters left running unintentionally.

Q7. What happens to packages installed using %pip when a cluster restarts?

A. They are lost, since clusters are ephemeral. Production teams typically manage dependencies through cluster-scoped libraries or requirements files instead of relying on ad-hoc %pip installs.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab